Intermediate (build real memory) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `conversation_memory_practice.py`: an LLM API call has no memory of its own — every call is stateless. "Memory" is really just resending the whole conversation (the full `messages` list of every user and assistant turn so far) on every single call. Building that resend-everything loop by hand, once, here, is what makes the Build Task's `chat_client.py` obvious instead of magic. **If not:** you'd be handed `send_message(history, user_input)` in the Build Task without ever having seen why it needs the whole `history` list passed in every time, not just the newest message.

Solution

```
# conversation_memory_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

# The whole trick: send the entire history every time, and get back just
# the new reply. This is the shape the Build Task's send_message() needs.
def get_reply(history):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    return response.choices[0].message.content


# A simple chat loop. The model has no memory of its own, so every earlier
# turn is resent in `history` on every call.
history = [{"role": "system", "content": "You are a helpful assistant."}]

while True:
    user_input = input("You: ")
    if user_input == "quit":
        break
    history.append({"role": "user", "content": user_input})
```

```
    reply = get_reply(history)
    history.append({"role": "assistant", "content": reply})
    print("Assistant:", reply)
```

Expected output: a running conversation where the assistant remembers earlier turns — type `quit` to stop.